SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- COMPARATIVE ANALYSIS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL2- COMPARATIVE ANALYSIS**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	*Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.*		
Student Name:	**Jannathul Firdaus N**	**Reg.No.:**	**192573025**
Department:	**CSE - DS**	**Date:**	**22.08.2026**

ANNEXURE A
COMPARATIVE ANALYSIS

1.Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.
2.Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.
3.Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.
4.Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.
5.Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Code of Conduct:

*I **Jannathul Firdaus N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Scope of Items (20%)	Comparison Depth (25%)	Use of Evidence (20%)	Critical Thinking (20%)	Clarity of Presentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: L1, L2, and L3 Cache Comparison

Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.

1. Scope of Items Compared

The comparison scope covers the three on-chip cache levels found in virtually all modern multicore processors, evaluated across three performance dimensions relevant to CO4: access latency (cycles to service a hit), bandwidth (data delivered per unit time), and throughput (sustained useful work delivered to the core, i.e., effective hit contribution to IPC). These items are chosen because they directly determine how close the memory subsystem comes to matching core execution speed, which is the central design tension in cache hierarchy engineering.

2. Comparison Table (Depth & Evidence)

Parameter	L1 Cache	L2 Cache	L3 Cache
Typical Size	32–64 KB (per core, split I/D)	256 KB–1 MB (per core/cluster)	2–32 MB (shared, last-level)
Access Latency	1–4 cycles	8–15 cycles	30–50 cycles
Bandwidth	Highest (multiple ports, adjacent to execution units)	High, but shared port limits some concurrency	Lower per-request bandwidth, but aggregated across many cores
Throughput Role	Sustains near-peak IPC for hot loops/data	Absorbs L1 misses without full DRAM penalty	Absorbs cross-core misses, reduces DRAM traffic
Scope	Private, per-core	Private or per-cluster	Shared, whole-chip

3. Critical Thinking / Interpretation

These figures are consistent with published microarchitecture data — e.g., Intel Core and AMD Zen designs document L1 hit latency around 4–5 cycles, L2 around 12–14 cycles, and shared L3 around 30–40+ cycles, with L1 bandwidth measured in hundreds of GB/s per core versus L3's aggregate bandwidth being shared across all cores contending for it.

4. Summary

The three-way trade-off is fundamentally one of size versus speed versus sharing scope: L1 is fast because it is small and private, which is precisely why it cannot hold enough data to avoid frequent misses on its own. L3, conversely, is large enough to capture cross-core sharing patterns (useful for multicore workloads with shared data structures) but its latency is high enough that relying on it alone would starve a modern core's execution pipeline.

The performance impact compounds multiplicatively across levels: a single L1 miss that hits in L2 costs roughly 3–4x the L1 latency; an L2 miss serviced by L3 costs another 3–4x; and an L3 miss serviced by DRAM costs an order of magnitude more again. This is why cache hierarchy design (inclusive vs. exclusive policies, replacement algorithms, prefetching) has a larger effect on real-world IPC than raw clock frequency increases in most modern workloads — the hierarchy determines how often the core actually has to pay the full DRAM penalty.

Question 2: SRAM vs DRAM

Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.

1. Scope of Items Compared

The scope compares SRAM and DRAM as the two dominant volatile memory technologies used respectively for on-chip caches and main memory, examined on latency, bandwidth, and throughput, plus the underlying circuit-level reasons (cell structure, refresh requirement) that cause those differences — these are the items that justify why the industry uses SRAM for cache and DRAM for main memory rather than either technology alone.

2. Comparison Table (Depth & Evidence)

Parameter	SRAM	DRAM
Cell Structure	6-transistor (6T) flip-flop, no capacitor	1 transistor + 1 capacitor (1T1C)
Latency	~1–2 ns (nanosecond-class, no refresh)	~10–20 ns plus periodic refresh stalls
Bandwidth (per die area)	High per bit, but low density limits total on-chip capacity	Lower per-cell speed, but wide buses/channels give high aggregate bandwidth (tens of GB/s)
Throughput	Sustains near-every-cycle access for cache hits	Row-buffer/bank-conflict sensitive; throughput drops on random access patterns
Density / Cost per bit	Low density, high cost per bit (6 transistors/cell)	High density, low cost per bit (1 transistor/cell)
Power	No refresh power, but higher static leakage per cell	Continuous refresh power draw even when idle
Typical Use	L1/L2/L3 cache, register files	Main system memory (DDR4/DDR5, LPDDR5)

3. Critical Thinking / Interpretation

This matches standard computer-architecture references (e.g., Hennessy & Patterson): SRAM's bistable 6T cell holds state without refresh, giving deterministic sub-nanosecond-class access, while DRAM's single-capacitor 1T1C cell must be periodically refreshed (typically every ~64 ms) because the capacitor leaks charge, and this refresh activity is a well-documented source of DRAM's variable latency and bandwidth loss under heavy load.

4. Summary

The core trade-off is again density versus speed: SRAM's 6-transistor cell is roughly 6–10x larger per bit than DRAM's 1-transistor-1-capacitor cell, which is precisely why SRAM cannot economically scale to gigabyte-class main memory, while DRAM's smaller cell enables the high density needed for main memory but at the cost of needing an external capacitor-refresh mechanism that both consumes power and introduces access variability.

This explains the architectural division of labor: caches (which must be small but blazingly fast to keep pace with a multi-GHz core) are built from SRAM, while main memory (which must be large enough to hold entire program working sets) is built from DRAM, with the cache hierarchy existing specifically to hide DRAM's latency and refresh-related throughput variability from the processor pipeline.

Question 3: Virtual Memory with Paging vs Cache Memory

Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.

1. Scope of Items Compared

The scope compares two distinct but structurally analogous memory-management mechanisms: cache memory (a hardware-managed, transparent speed buffer between core and DRAM) and virtual memory with paging (an OS/hardware-cooperative mechanism that maps a large virtual address space onto limited physical DRAM, backed by disk/SSD). Both use similar principles — locality, hit/miss, replacement policy — making them directly comparable on access latency and system throughput despite operating at very different scales.

2. Comparison Table (Depth & Evidence)

Parameter	Cache Memory	Virtual Memory (Paging)
Managed By	Hardware (transparent to software)	OS + hardware (MMU, page tables) cooperatively
Hit Latency	1–40 cycles (L1–L3)	~100–300 cycles for a TLB-hit, in-memory page access
Miss Penalty	~100–300 cycles (DRAM access)	Page fault: milliseconds (disk-backed) — 5–6 orders

Parameter	Cache Memory	Virtual Memory (Paging)
		of magnitude slower than a cache miss
Granularity	Cache line (32–128 bytes)	Page (typically 4 KB, or 2 MB/1 GB huge pages)
Purpose	Hide DRAM latency from the CPU core	Extend addressable memory beyond physical DRAM; enable process isolation
Throughput Impact	High — determines effective IPC directly	Page faults severely degrade throughput; TLB misses add page-walk overhead even without faults

3. Critical Thinking / Interpretation

The orders-of-magnitude gap between a cache miss (serviced from DRAM in ~100 ns) and a page fault (serviced from disk/SSD in milliseconds) is a standard and well-evidenced figure in operating-systems and architecture literature, and is the reason OS designers treat page faults as exceptional, expensive events to be minimized, whereas cache misses are routine, tolerated events the hardware handles every cycle.

4. Summary

Both mechanisms apply the same underlying principle — a small, fast structure (cache line / TLB entry) sits in front of a larger, slower one (DRAM / disk) and relies on locality of reference to keep most accesses fast — but they operate at vastly different time scales and consequences: a cache miss merely adds tens to hundreds of nanoseconds, while a page fault can stall a process for milliseconds and forces an OS-level context switch to let other processes run during the fault-service time.

System throughput is therefore governed by different levers for each: cache throughput is improved by better locality-exploiting hardware (associativity, prefetching, replacement policy), while virtual-memory throughput is improved

chiefly by minimizing fault rate (adequate physical RAM, working-set-aware page replacement, huge pages to cut TLB-miss overhead) — conflating the two when tuning a system (e.g., trying to fix a page-fault problem by resizing cache) would be a category error.

Question 4: NAND Flash vs DRAM

Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.

1. Scope of Items Compared

The scope compares NAND Flash and DRAM as representative non-volatile-storage and volatile-main-memory technologies respectively, on latency, bandwidth, and — critically for CO4 — their suitability as a specific tier within a hierarchical memory system, since these two technologies sit at very different points in the latency/capacity/persistence trade-off space.

2. Comparison Table (Depth & Evidence)

Parameter	**NAND Flash**	**DRAM**
Volatility	Non-volatile (retains data without power)	Volatile (requires continuous refresh/power)
Read Latency	~10–100 μs (page read)	~10–20 ns
Write Latency	Higher still; requires block erase before rewrite (~ms for erase)	~10–20 ns, no erase-before-write needed
Bandwidth	Hundreds of MB/s to a few GB/s (interface-limited, e.g., NVMe)	Tens of GB/s (parallel DRAM channels)
Endurance	Limited program/erase cycles (wear leveling required)	Effectively unlimited write endurance
Density/Cost	Very high density, low cost per bit	Lower density, higher cost per bit than flash

Parameter	NAND Flash	DRAM
Hierarchical Role	Persistent storage tier (SSD) — bulk/cold data	Main memory tier — active working set

3. Critical Thinking / Interpretation

These figures align with standard SSD/DRAM datasheet comparisons: NAND Flash read latency (tens of microseconds) is roughly 1,000x slower than DRAM (tens of nanoseconds), and NVMe SSD interface bandwidth, while high for storage, remains well below DRAM channel bandwidth — this gap is the well-documented rationale for the DRAM-as-cache-for-flash pattern seen in modern OS page caches and SSD DRAM buffers.

4. Summary

The latency gap between NAND and DRAM is roughly three orders of magnitude, which means NAND cannot be used as a direct substitute for DRAM in any latency-sensitive role — its value lies entirely in non-volatility and density/cost, making it suited to the storage tier rather than the working-memory tier.

In a hierarchical memory system, this places NAND Flash below DRAM: DRAM holds the actively executing working set for fast CPU access, while NAND Flash provides large, cheap, persistent capacity for data not currently in use, with the OS/file system managing the movement of pages between the two tiers (paging/demand-loading) — the same latency-vs-capacity trade-off seen between cache and DRAM repeats itself one level down between DRAM and flash.

Question 5: MRAM vs RRAM

Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

1. Scope of Items Compared

The scope compares MRAM (Magnetoresistive RAM) and RRAM (Resistive RAM), two leading emerging non-volatile memory technologies positioned to bridge the gap between DRAM's speed and Flash's non-volatility, evaluated on access time, throughput, and energy efficiency — the dimensions most relevant to their candidacy as new tiers in next-generation hierarchical memory systems.

2. Comparison Table (Depth & Evidence)

Parameter	**MRAM**	**RRAM (ReRAM)**
Storage Mechanism	Magnetic tunnel junction (MTJ); resistance set by electron spin orientation	Resistance change in a metal-oxide dielectric via formation/rupture of conductive filaments
Access Time (Read)	~10–30 ns (near-DRAM speed)	~10–50 ns (comparable, slightly more variable)
Write Endurance	Very high (10^12–10^15 cycles), near-DRAM-class	Moderate (10^6–10^12 cycles depending on process), lower than MRAM
Throughput	High, symmetric read/write speed suited to frequent updates	High read throughput; write throughput can be limited by filament-forming variability
Energy Efficiency	Low read energy; write energy moderate (current-driven switching)	Very low read/write energy per bit; well suited to compute-in-memory
Scalability/Density	Good, but MTJ stack adds process complexity	Excellent scalability, simple two-terminal structure enables high density

Parameter	MRAM	RRAM (ReRAM)
Best Fit	Fast non-volatile cache/register replacement, frequent-write applications	Dense non-volatile storage tier, in-memory computing / AI inference weights

3. Critical Thinking / Interpretation

These characteristics are consistent with published emerging-memory surveys: MRAM (specifically STT-MRAM) is widely reported to offer DRAM-comparable access latency with very high write endurance due to its non-degrading magnetic switching mechanism, whereas RRAM's simpler two-terminal metal-oxide structure gives it a density and scaling advantage but with somewhat lower and more process-dependent endurance because filament formation gradually degrades the dielectric.

4. Summary

The key differentiator is the physical switching mechanism: MRAM's magnetic-spin-based storage does not physically degrade the memory cell on each write, which is why it achieves near-unlimited endurance and is attractive wherever data is updated very frequently (e.g., as a persistent cache or working register replacement); RRAM's filament-based resistive switching does incur gradual material wear, capping its endurance lower, but its simpler structure and smaller cell footprint make it more scalable and cost-effective for high-density, less write-intensive roles such as storing largely-static AI model weights or acting as a fast-loading non-volatile storage tier.

From an energy-efficiency standpoint, RRAM's low switching energy and compatibility with analog compute-in-memory (performing multiply-accumulate operations directly within the memory array) gives it a distinct advantage for emerging AI/edge workloads where data movement dominates energy consumption, while MRAM's advantage is in reliability and endurance for frequently-updated, latency-sensitive state.